



Find all Duplicate Numbers (easy)

We'll cover the following ^

- Problem Statement
- Try it yourself
 - Solution
 - Code
 - Time complexity
 - Space complexity

Problem Statement

We are given an unsorted array containing n numbers taken from the range 1 to n . The array has some numbers appearing twice, **find all these duplicate numbers** using constant space.

Example 1:

Input: [3, 4, 4, 5, 5]
Output: [4, 5]

Example 2:

Input: [5, 4, 7, 2, 3, 5, 3]
Output: [3, 5]

Try it yourself

Try solving this question here:

Java

Python3

JS

C++

```
1 import java.util.*;
2
3 class FindAllDuplicate {
4
5     public static List<Integer> findNumbers(int[] nums) {
6         List<Integer> duplicateNumbers = new ArrayList<>();
7         // TODO: Write your code here
8         return duplicateNumbers;
9     }
10 }
11
```



Test

Save

Reset



Solution

This problem follows the **Cyclic Sort** pattern and shares similarities with [Find the Duplicate Number](#). Following a similar approach, we will place each number at its correct index. After that, we will iterate through the array to find all numbers that are not at the correct indices. All these numbers are duplicates.

Code

Here is what our algorithm will look like:

Java

Python3

C++

JS

```
1  import java.util.*;
2
3  class FindAllDuplicate {
4
5      public static List<Integer> findNumbers(int[] nums) {
6          int i = 0;
7          while (i < nums.length) {
8              if (nums[i] != nums[nums[i] - 1])
9                  swap(nums, i, nums[i] - 1);
10             else
11                 i++;
12         }
13
14         List<Integer> duplicateNumbers = new ArrayList<>();
15         for (i = 0; i < nums.length; i++) {
16             if (nums[i] != i + 1)
17                 duplicateNumbers.add(nums[i]);
18         }
19
20         return duplicateNumbers;
21     }
22
23     private static void swap(int[] arr, int i, int j) {
24         int temp = arr[i];
25         arr[i] = arr[j];
26         arr[j] = temp;
27     }
28
29     public static void main(String[] args) {
30         List<Integer> duplicates = FindAllDuplicate.findNumbers(new int[] { 3, 4, 4, 5, 5 });
31         System.out.println("Duplicates are: " + duplicates);
32
33         duplicates = FindAllDuplicate.findNumbers(new int[] { 5, 4, 7, 2, 3, 5, 3 });
34         System.out.println("Duplicates are: " + duplicates);
35     }
36 }
37
```



Run

Save

Reset



Time complexity

The time complexity of the above algorithm is $O(n)$.



Space complexity

Ignoring the space required for storing the duplicates, the algorithm runs in constant space $O(1)$.

[← Back](#)[✓ Completed](#)[Next →](#)

Find the Duplicate Number (easy)

Problem Challenge 1

